

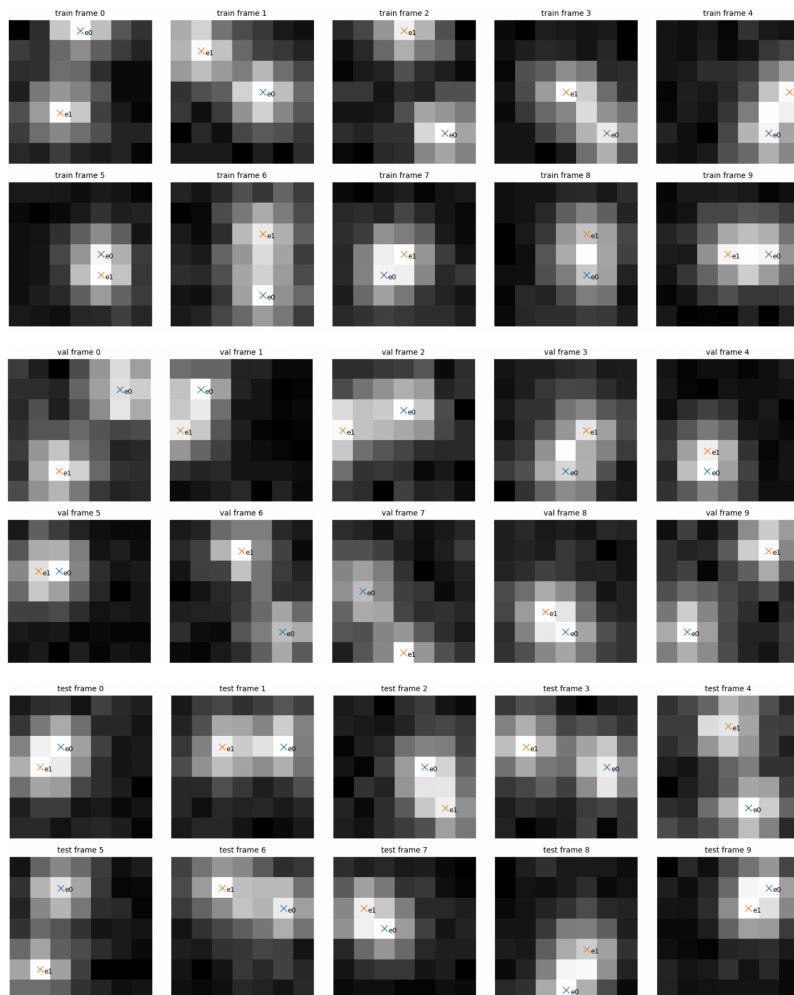
In this document I provided the result of pure KAN and CKAN on 2 types of data frames.

First I tried both networks on 7*7 (2 emitters per frame), the result was good. So then I generated 64*64 (10 emitters per frame) and also the result is very good for these dataframes too.

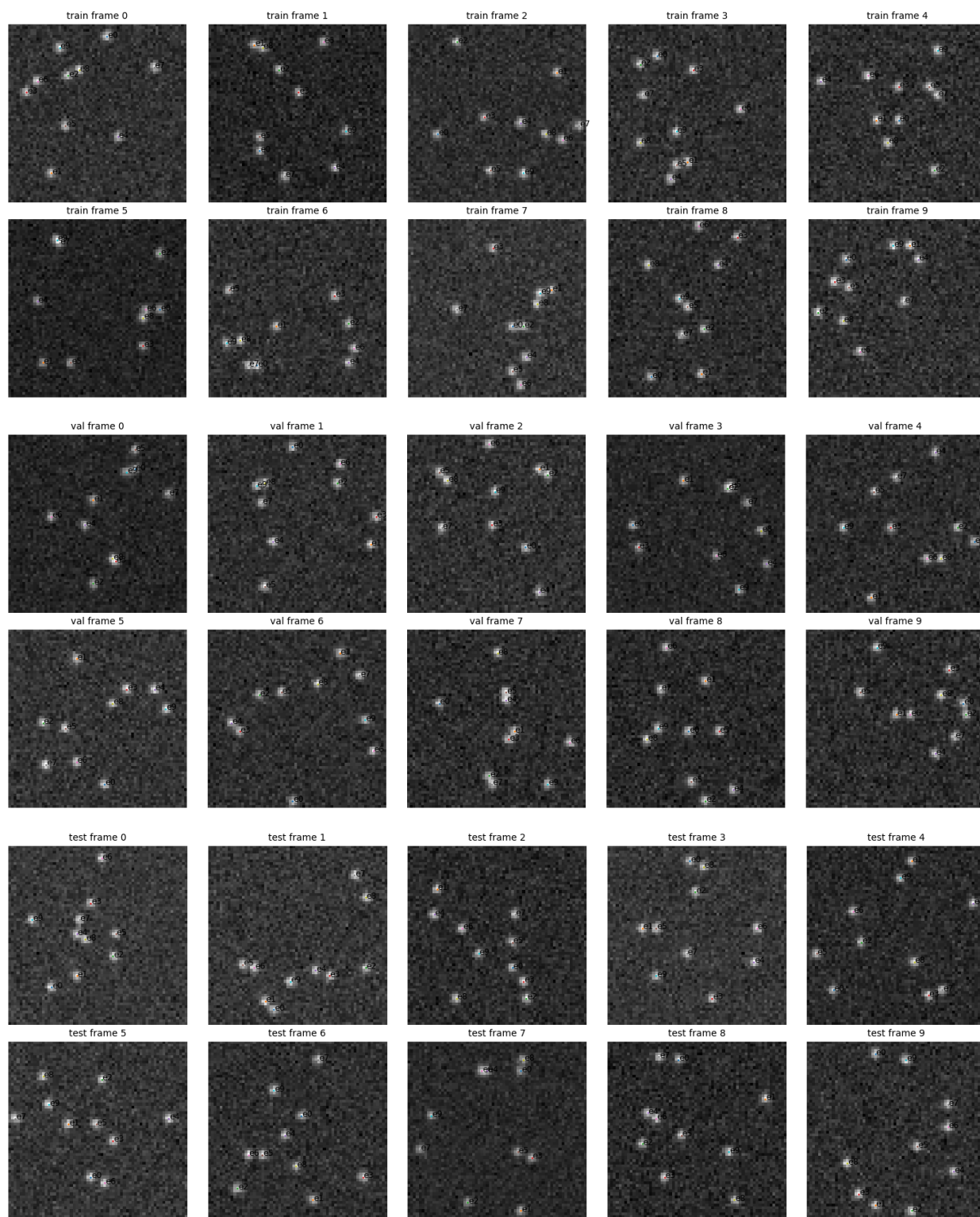
The point is that according to what we have talked about in the last meetings the emitters are in the center of each pixel.

Dataframes:

1. 7*7 px patches (each pixel 100*100 nm) - 2 emitters per frame - emitters in the center



2. 64*64 px patches (each pixel 100*100 nm) - 10 emitters per frame - emitters in the center



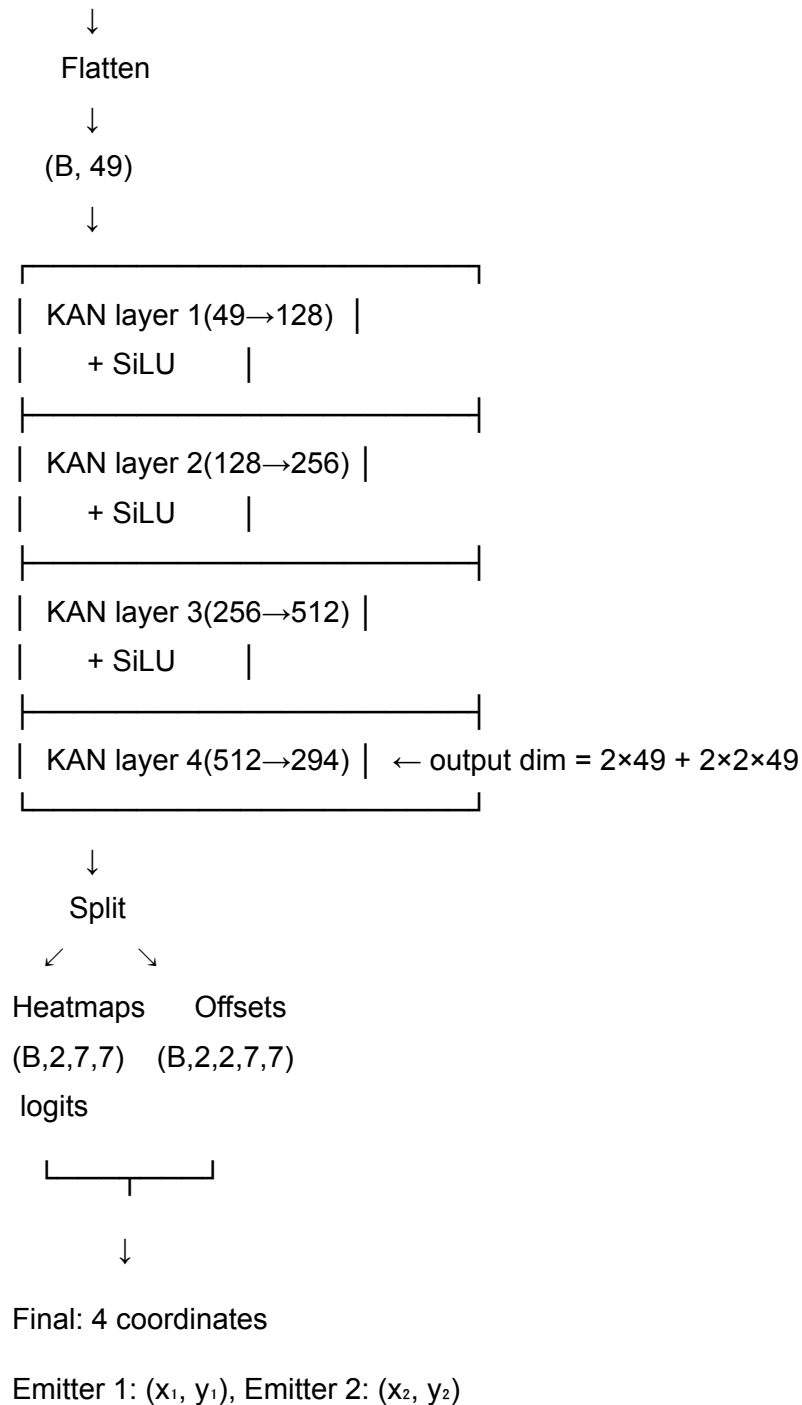
Pure KAN for Single-Molecule Localization Microscopy (version 11)

Network Architecture: PureKAN localization in SMLM

order-k B-spline= 3

grid=8

Input: (B, 1, 7, 7) (raw intensity values from frames)



Predicted coordinate calculation:

$$\hat{x} = p_x + 0.5 + \hat{dx}, \quad \hat{y} = p_y + 0.5 + \hat{dy}$$

Output:

1. Heatmap logits: (B,2,7,7): It is producing a score for each pixel (a heatmap), so it can decide which pixel is the emitter location.

- One 7×7 probability map per emitter
- Represents which pixel contains the emitter

2. Offset maps: (B,2,2,7,7)

- For each emitter, for each pixel: predicted (dx,dy)
- Only the offset at the selected pixel is used during loss calculation

Output: 294 dimensions

└─ Heatmaps: 98 values

└─ Emitter 1: 49 logits (7×7 grid)

└─ Emitter 2: 49 logits (7×7 grid)

└─ Offsets: 196 values

Emitter 1: 98 values └─ dx: 49 values (one per pixel) └─ dy: 49 values (one per pixel) |

Emitter 2: 98 values └─ dx: 49 values (one per pixel) └─ dy: 49 values (one per pixel)

Loss function:

$$L = W(\text{CE}) * L(\text{CE}) + W(\text{loc}) * L(\text{loc})$$

Cross entropy loss and localization loss

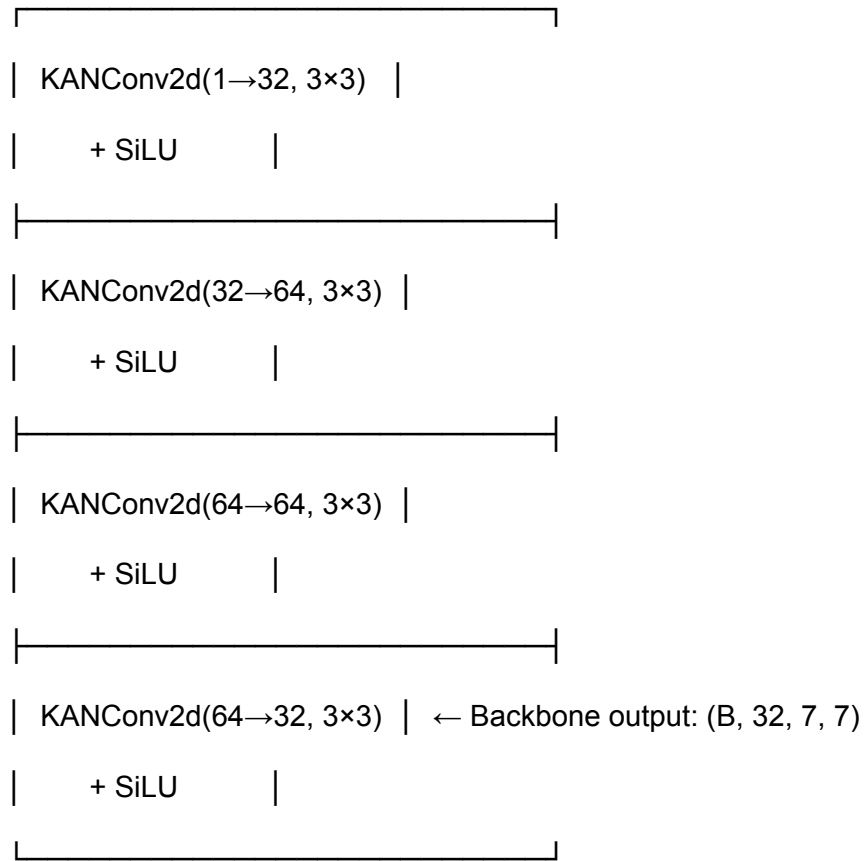
Note: offset predicts deviation from center: $\hat{dx} \in [-0.5, 0.5]$

CKAN (Convolutional KAN) for SMLM (version 3)

Network Architecture:

Input: (B, 1, 7, 7)

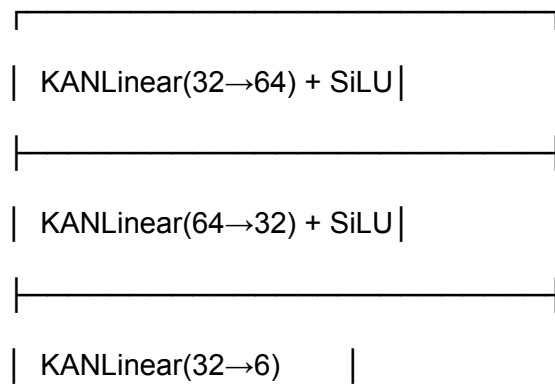
↓

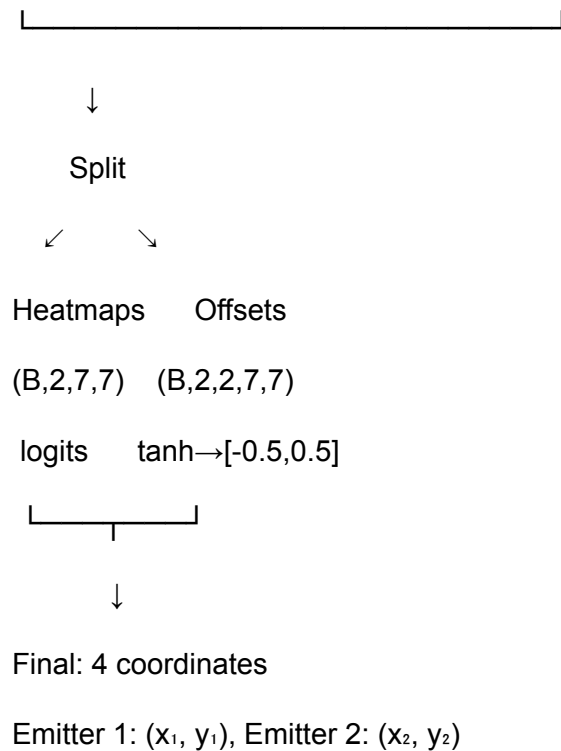


↓

Reshape to (B×49, 32) ← Per-pixel feature vectors

↓





Two-Stage Design

Stage 1: Convolutional Backbone

The KANConv2d layers act as a feature extractor that:

- Learns spatial patterns (PSF shapes, intensity gradients)
- Uses learnable activations to adapt to the dataframe

Stage 2: Per-Pixel KAN Head

After the backbone, each pixel position has a 32-dimensional feature vector. The KANLinear head processes each pixel independently.

Heatmaps: $(B, 2, 7, 7)$

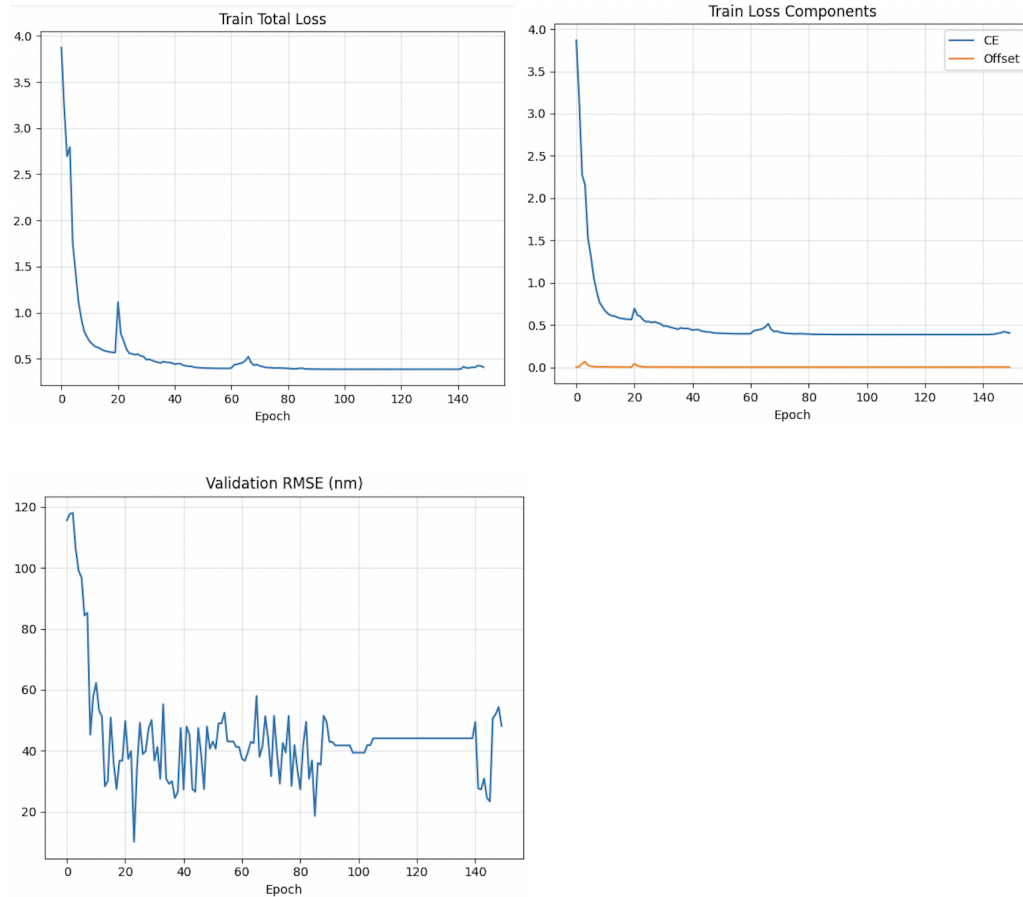
- Logits for 49-class classification per emitter
- Softmax over spatial dimensions gives probability map

Offsets: $(B, 2, 2, 7, 7)$

- Shape meaning: $[batch, emitter, (dx, dy), height, width]$
- Bounded to $[-0.5, 0.5]$ via $0.5 \cdot \tanh(\cdot)$

Result:

CKAN - 7*7 dataframe



TEST SET RESULTS (CKAN)

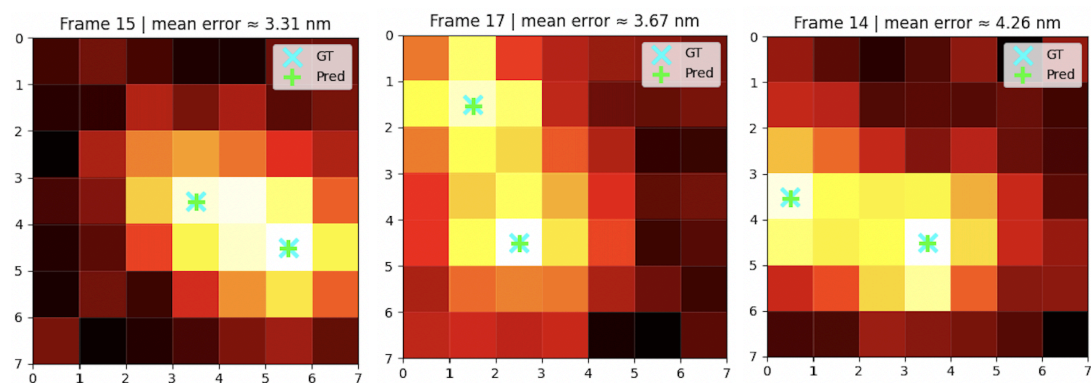
=====

Pixel Accuracy: 94.00%

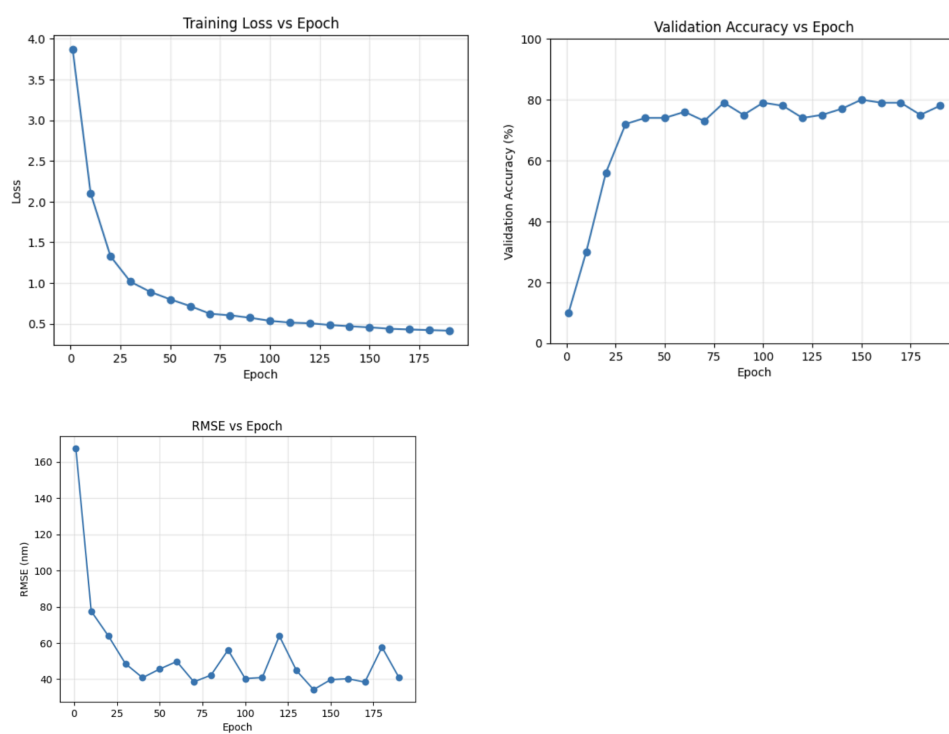
Per-emitter Acc: E1=96.00%, E2=92.00%

RMSE: 0.2455 px (24.55 nm)

=====



Pure kAN 7*7 dataframe



RMSMD table

Network	RMSMD (7*7dataframe) 2 emitters	RMSMD (64*64 dataframe) 10 emitters
CKAN	1.010 nm	21.635 nm
Pure KAN	6.242 nm	22.636 nm

Detailed Analysis

1. Direct Regression KAN:

- Asks the network to learn a single, complex mapping from all pixels to exact coordinates
- Must simultaneously learn: (a) where emitters are, (b) exact sub-pixel position

Raw pixels → Flatten → KAN layers → Sigmoid → Scale to $[0, H] \times [0, W]$

2. Patch-Based KAN:

Decomposes into two well-defined sub-tasks:

1. Classification: Which pixel contains the emitter?
2. Regression: Where within that pixel?

Each sub-task is dramatically simpler than the combined task

The KAN outputs **3 values per emitter** at each pixel location:

- **1 logit** for heatmap (classification)
- **2 offsets** (dx, dy) for sub-pixel refinement

Network	RMSMD (7*7dataframe) 2 emitters	RMSMD (64*64 dataframe) 10 emitters
KAN(whole dataframe)	28.843	48.092
KAN(patch based)	6.242 nm	22.636 nm

With all the versions and networks I implemented in KAN that's the final conclusion:

1. If we use patch based (cropping) and give the network one emitter patch KAN can localize emitter with high accuracy, but the number of parameters vary based on the dataframe size but the speed of running is more than the second approach.
2. If we feed the network with the whole frame the accuracy is not high compared with theory and CKAN and patch based approach. The number of parameters vary but the time of running is less than the first approach.

Approach 1: Patch-Based KAN

- **Method:** Extract local $K \times K$ patches around each pixel, apply KAN to predict heatmap (which pixel) + offsets (sub-pixel refinement)
- **Accuracy:** High — achieves results comparable to or approaching theoretical limits and CKAN
- **Parameters:** Relatively small and constant (~280K), independent of frame size
- **Speed:** Slower per epoch due to processing all pixel patches, but requires fewer epochs to converge
- **Conclusion :** Decomposes localization into two easier sub-tasks (classification + bounded regression), preserves spatial locality matching PSF physics, and benefits from dense supervision

Approach 2: localization KAN

- **Method:** Flatten entire frame, feed through KAN layers, directly regress emitter coordinates
- **Accuracy:** Low — significantly underperforms compared to patch-based approach, CKAN, and theoretical limits
- **Parameters:** Scales with frame size (can explode to millions for larger frames)
- For example for 64×64 (10 emitters) -> Parameters: 36,217,752 total
- **Speed:** Faster per epoch (single forward pass), but accuracy remains poor even with more training
- **Conclusion:** Destroys spatial structure by flattening, attempts to solve a harder optimization problem but still proves KAN can work!